

FUNDAMENTOS DE PYTHON 1

32 HORAS

ROTEIRO DO CURSO

1 Introdução ao Python

2 Operações de Entrada/Saída

3 Estruturas Condicionais e de Repetição

4 Funções e Estruturas de Dados

OBJETIVOS DA AULA

1. Conhecer as tuplas.
2. Conhecer os dicionários.
3. Manipular elementos de dicionários.

CONHECER AS TUPLAS

Uma tupla em Python é uma sequência de objetos imutáveis, ou seja, uma vez criada não pode ser modificada. Não se pode adicionar ou remover elementos dela.



São usadas para armazenar coleções de itens relacionados, que não devem ser alterados após a criação.

Normalmente são usadas para guardar informações protegidas, no intuito de prevenir modificações acidentais.

CONHECER AS TUPLAS

Quando usar tuplas em Python? Quais são seus cenários de uso?

IMUTABILIDADE	DESEMPENHO	CHAVES DE DICIONÁRIOS	SEGURANÇA
Não podem ser alteradas, portanto, previnem a manipulação de informações	São mais eficientes no uso de memória e em velocidade, pois não possuem tantas funções e operações	Podem ser usadas como chaves (cabeçalho) em dicionários, por sua natureza imutável	Podem ser usadas em operações que requerem segurança, por não poderem ser modificadas

CONHECER AS TUPLAS

TUPLA DECLARAÇÃO

Para criar uma tupla, é necessário usar **parênteses ()** e inserir os elementos separados, dentro deles, por vírgulas.

```
1  # Cria uma tupla numérica.  
2  numeros = (1, 2, 3, 4, 5)  
3  
4  # Cria uma tupla de texto.  
5  cores = ("Vermelho", "Verde", "Azul")  
6  
7  # Cria uma tupla vazia.  
8  vazia = ()
```

CONHECER AS TUPLAS

TUPLA

ACESSAR OS ELEMENTOS

Os elementos de uma tupla são acessados através de índices em ordem ascendente (0, 1 e 2), descendente (-1, -2 e -3), ou de acordo com um intervalo de índices (2:3 ou 1:5).

```
1  # Cria uma tupla numérica.
2  tupla = (10, 20, 30, 40, 50)
3
4  # Exibe o primeiro elemento da tupla.
5  print(tupla[0])
6
7  # Exibe o último elemento da tupla.
8  print(tupla[-1])
9
10 # Exibe os elementos de 1 a 4 (exceto 4).
11 print(tupla[1:4])
```

CONHECER AS TUPLAS

TUPLA AGRUPADA DECLARAÇÃO

Uma tupla agrupada é uma tupla que contém outras tuplas como seus elementos (agrupada uma dentro da outra). Cada tupla pode representar linhas e colunas.

```
1  # Cria duas tuplas.  
2  numerica = (10, 20, 30)  
3  texto = ("Um", "Dois", "Três")  
4  
5  # Faz o agrupamento das duplas.  
6  agrupada = (numerica, texto)  
7  
8  # Exibe a tupla agrupada.  
9  print(agrupada)
```

CONHECER AS TUPLAS

TUPLA AGRUPADA ACESSAR OS ELEMENTOS

Para acessar os elementos de uma tupla agrupada é necessário usar dois índices, o primeiro para selecionar a linha e o segundo para selecionar a coluna que se deseja, da tupla.

```
2  agrupada = (  
3      (10, 20, 30),  
4      (40, 50, 60),  
5      (70, 80, 90)  
6  )  
7  
8  # Exibe o 2º elemento da 1ª tupla.  
9  print(agrupada[0][1])  
10  
11 # Exibe o 2º elemento da 2ª tupla.  
12 print(agrupada[1][1])
```



CONHECER OS DICIONÁRIOS

CONHECER OS DICIONÁRIOS

Em Python, um dicionário é uma estrutura de dados que armazena pares de chave-valor de forma associativa. Cada chave é única e mapeia um valor específico.



- Os dicionários são mutáveis, o que significa que se pode adicionar, remover e modificar entradas após criá-las.
- São usados para guardar e acessar dados de maneira eficiente, usando uma chave para recuperar um valor associado.

CONHECER OS DICIONÁRIOS

Portanto, sua estrutura é composta por:



CHAVE

É um valor único que atua como um identificador no dicionário. Pode-se usar **strings**, **inteiros** e **tuplas** como chaves, pois são imutáveis.



VALOR

É o dado associado a uma chave. Os valores podem ser **strings**, **números inteiro** ou **decimais**, **listas**, **outros dicionários** e até mesmo funções.

CONHECER OS DICIONÁRIOS

CRIAR UM DICIONÁRIO

A forma mais comum de criar um dicionário é usando um par de **chaves** `{}` para definir o início e fim da estrutura...

```
1  # Cria um dicionário da maneira mais simples.
2  d1 = {'nome': 'Marcus', 'sobrenome': 'Vinícius', 'idade': '27 anos'}
3
4  # Cria um dicionário utilizando uma função embutida.
5  d2 = dict(nome='Marcus', sobrenome='Vinícius', idade='27 anos')
6
7  # Cria um dicionário concatenando os pares de chave/valor.
8  d3 = dict(zip(['nome', 'sobrenome', 'idade'], ['Marcus', 'Vinícius', '27 anos']))
```

CONHECER OS DICIONÁRIOS

ACESSAR OS ITENS DE UM DICIONÁRIO

É necessário informar o nome do dicionário existente e a chave associada ao campo que quer acessar...

```
1  # Cria um dicionário para armazenar informações de uma pessoa.
2  pessoa = {'nome': 'Marcus Vinícius', 'altura': 1.75, 'nascimento': '11/02/1996'}
3
4  # Acessa as informações da pessoa no dicionário.
5  print(pessoa['nome'])
6  print(pessoa['altura'])
7  print(pessoa['nascimento'])
```



MANIPULAR ELEMENTOS DE UM DICIONÁRIO

MANIPULAR ELEMENTOS DE UM DICIONÁRIO

OBTENÇÃO DAS CHAVES E VALORES

Os métodos **KEYS** e **VALUES** podem ser usados para retornar todas as chaves e valores de um dicionário...

```
1  # Cria um dicionário para armazenar informações de uma pessoa.
2  pessoa = {'nome': 'Marcus Vinícius', 'altura': 1.75, 'nascimento': '11/02/1996'}
3
4  # Obtém as chaves do dicionário.
5  print(pessoa.keys())
6
7  # Obtém os valores do dicionário.
8  print(pessoa.values())
```

MANIPULAR ELEMENTOS DE UM DICIONÁRIO

PERCORRER OS ITENS

O método **ITEMS** pode ser usado para percorrer e exibir os valores e chaves de um dicionário em um loop...

```
1  # Cria um dicionário para armazenar informações de uma pessoa.
2  pessoa = {'nome': 'Marcus Vinícius', 'altura': 1.75, 'nascimento': '11/02/1996'}
3
4  # Percorre, em um laço de repetição, todos os itens contidos em um dicionário.
5  for chave, valor in pessoa.items():
6      print(f"Chave: {chave} e Valor: {valor}")
```

MANIPULAR ELEMENTOS DE UM DICIONÁRIO

VERIFICAR SE EXISTE UMA CHAVE OU VALOR

O método **IN** com os métodos **KEYS** e **VALUES** pode ser usado para verificar a existência de uma chave ou valor...

```
1 # Cria um dicionário para armazenar tonalidades de cores.
2 cores = {'vermelho': 255, 'verde': 0, 'azul': 100}
3
4 # Verifica se uma chave existe no dicionário.
5 if ('verde' in cores.keys()): print("A cor é uma chave!!!")
6
7 # Verifica se um valor existe no dicionário.
8 if (255 in cores.values()): print("O valor existe!!!")
```

MANIPULAR ELEMENTOS DE UM DICIONÁRIO

ADICIONAR ELEMENTOS

Para adicionar um novo item em um dicionário deve-se criar uma nova chave...

```
1  # Cria um dicionário para armazenar informações de uma pessoa.
2  pessoa = {'nome': 'Marcus Vinícius', 'altura': 1.75, 'nascimento': '11/02/1996'}
3
4  # Adiciona um novo item ao dicionário criado anteriormente.
5  pessoa['cpf'] = '000.000.000-00'
6
7  # Exibe o dicionário com todos os itens.
8  print(pessoa)
```

MANIPULAR ELEMENTOS DE UM DICIONÁRIO

ATUALIZAR ELEMENTOS

O método **UPDATE** pode ser usado para atualizar, ou criar, um item novo, caso não exista, em um dicionário...

```
1  # Cria um dicionário para armazenar informações de uma pessoa.  
2  pessoa = {'nome': 'Marcus Vinícius', 'altura': 1.75}  
3  
4  # Atualiza um item do dicionário existente.  
5  pessoa.update({'nome': 'Ronaldo Nazário'})
```

MANIPULAR ELEMENTOS DE UM DICIONÁRIO

EXCLUIR ELEMENTOS

Os métodos **DEL** e **POP** podem ser usados para excluir um item em um dicionário...

```
1  # Cria um dicionário para armazenar informações de uma pessoa.
2  pessoa = {'nome': 'Marcus Vinícius', 'altura': 1.75, 'nascimento': '11/02/1996'}
3
4  # Excluí um item de um dicionário a partir do método DEL.
5  del pessoa['nascimento']
6
7  # Excluí um item de um dicionário a partir do método POP.
8  pessoa.pop('altura')
```